

Synchronous FIFO



Prepared by Bassam Hussam

Under supervision of

Eng. Kareem Waseem

Specifications:

Parameters

- FIFO_WIDTH: DATA in/out and memory word width (default: 16)
- FIFO_DEPTH: Memory depth (default: 8)

Ports

Port	Direction	Function
data_in	Input	Write Data: The input data bus used when writing the FIFO.
wr_en		Write Enable: If the FIFO is not full, asserting this signal causes data (on data_in) to be written into the FIFO
rd_en		Read Enable: If the FIFO is not empty, asserting this signal causes data (on data_out) to be read from the FIFO
clk		Clock signal
rst_n		Active low asynchronous reset
data_out	Output	Read Data: The sequential output data bus used when reading from the FIFO.
full		Full Flag: When asserted, this combinational output signal indicates that the FIFO is full. Write requests are ignored when the FIFO is full, initiating a write when the FIFO is full is not destructive to the contents of the FIFO.
almostfull		Almost Full: When asserted, this combinational output signal indicates that only one more write can be performed before the FIFO is full.
empty		Empty Flag: When asserted, this combinational output signal indicates that the FIFO is empty. Read requests are ignored when the FIFO is empty, initiating a read while empty is not destructive to the FIFO.
almostempty		Almost Empty: When asserted, this output combinational signal indicates that only one more read can be performed before the FIFO goes to empty.
overflow		Overflow: This sequential output signal indicates that a write request (wr_en) was rejected because the FIFO is full. Overflowing the FIFO is not destructive to the contents of the FIFO.
underflow		Underflow: This sequential output signal Indicates that the read request (rd_en) was rejected because the FIFO is empty. Under flowing the FIFO is not destructive to the FIFO.
wr_ack		Write Acknowledge: This sequential output signal indicates that a write request (wr_en) has succeeded.

Design before fixing:

```
module FIFO(data_in, wr_en, rd_en, clk, rst_n, full, empty, almostfull, almostempty,
wr_ack, overflow, underflow, data_out);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
input [FIFO_WIDTH-1:0] data_in;
input clk, rst_n, wr_en, rd_en;
output reg [FIFO_WIDTH-1:0] data_out;
output reg wr_ack, overflow;
output full, empty, almostfull, almostempty, underflow;
localparam max_fifo_addr = $clog2(FIFO_DEPTH);
reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_ptr <= 0;
    end
    else if (wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= data_in;
        wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        wr_ack <= 0;
        if (full & wr_en)
            overflow <= 1;
        else
            overflow <= 0;
    end
end
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_ptr <= 0;
    end
    else if (rd_en && count != 0) begin
        data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
end
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({wr_en, rd_en} == 2'b10) && !full)
            count <= count + 1;
        else if ( ({wr_en, rd_en} == 2'b01) && !empty)
            count <= count - 1;
    end
end
end
```

```

assign full = (count == FIFO_DEPTH)? 1 : 0;
assign empty = (count == 0)? 1 : 0;
assign underflow = (empty && rd_en)? 1 : 0;
assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;
assign almostempty = (count == 1)? 1 : 0;
endmodule

```

Design after fixing and Assertions:

Bugs:

- The 'overflow' and 'wr_ack' should be reset.
- The underflow signal is supposed to be sequential output not combinational and changing its type to output only not reg.
- The almost full signal is supposed to be high when the count is below the depth of the FIFO by one not two.
- The count should check also for cases when the write and read enables are high while it's empty or full, as one operation should take place then the counter will increment or decrement.

Code:

```

module FIFO(Sync_FIFO_interface.DUT v_if);
localparam max_fifo_addr = $clog2(v_if.FIFO_DEPTH);
logic [v_if.FIFO_WIDTH-1:0] mem [v_if.FIFO_DEPTH-1:0];
logic [max_fifo_addr-1:0] wr_ptr, rd_ptr;
logic [max_fifo_addr:0] count;
always @(posedge v_if.clk or negedge v_if.rst_n) begin
    if (!v_if.rst_n) begin
        wr_ptr <= 0;
        v_if.wr_ack <= 0; //was bug
        v_if.overflow <= 0; //was bug
    end
    else if (v_if.wr_en && !v_if.full) begin
        mem[wr_ptr] <= v_if.data_in;
        v_if.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        v_if.wr_ack <= 0;
        if (v_if.full && v_if.wr_en)
            v_if.overflow <= 1;
        else
            v_if.overflow <= 0;
    end
end
always @(posedge v_if.clk or negedge v_if.rst_n) begin
    if (!v_if.rst_n) begin
        rd_ptr <= 0;
        v_if.underflow <= 0; //was bug
    end
end

```

```

end
else if (v_if.rd_en && !v_if.empty) begin
    v_if.data_out <= mem[rd_ptr];
    rd_ptr <= rd_ptr + 1;
end
else begin //was bug
    if (v_if.empty && v_if.rd_en)
        v_if.underflow <= 1;
    else
        v_if.underflow <=0;
    end
end
end
always @(posedge v_if.clk or negedge v_if.rst_n) begin
    if (!v_if.rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({v_if.wr_en, v_if.rd_en} == 2'b10) && !v_if.full)
            count <= count + 1;
        else if ( ({v_if.wr_en, v_if.rd_en} == 2'b01) && !v_if.empty)
            count <= count - 1;
        else if(({v_if.wr_en, v_if.rd_en} == 2'b11) && v_if.full) //was bug
            count <= count-1;
        else if(({v_if.wr_en, v_if.rd_en} == 2'b11) && v_if.empty) //bug continue
            count <= count+1;
    end
end
end
assign v_if.full = (count == v_if.FIFO_DEPTH)? 1 : 0;
assign v_if.empty = (count == 0)? 1 : 0;
assign v_if.almostfull = (count == v_if.FIFO_DEPTH-1)? 1 : 0; //was bug
assign v_if.almostempty = (count == 1)? 1 : 0;
////////////////////////////////////
//////////////////////////////////// Assertions////////////////////////////////////
property reset_internal;
    @(negedge v_if.clk) !v_if.rst_n
    |-> ({count,rd_ptr,wr_ptr}==3'b0) ;
endproperty
assert property (reset_internal);
cover property (reset_internal);

property write_acknowledge_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
    (v_if.wr_en)&&(!v_if.full) |=> v_if.wr_ack ;
endproperty
assert property (write_acknowledge_p);
cover property (write_acknowledge_p);

property overflow_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
    (v_if.wr_en && v_if.full) |=> v_if.overflow;
endproperty
assert property (overflow_p);
cover property (overflow_p);

```

```

property underflow_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
        (v_if.rd_en && v_if.empty) | => v_if.underflow;
endproperty

assert property (underflow_p);
cover property (underflow_p);

property empty_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
        (count==0) |-> v_if.empty;
endproperty

assert property (empty_p);
cover property (empty_p);

property full_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
        (count==v_if.FIFO_DEPTH) |-> v_if.full;
endproperty

assert property (full_p);
cover property (full_p);

property almostfull_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
        (count==v_if.FIFO_DEPTH-1) |-> v_if.almostfull;
endproperty

assert property (almostfull_p);
cover property (almostfull_p);

property almostempty_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
        (count==1) |-> v_if.almostempty;
endproperty

assert property (almostempty_p);
cover property (almostempty_p);

property read_ptr_wrap_around_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
        (rd_ptr==v_if.FIFO_DEPTH-1) && !v_if.empty && v_if.rd_en | => (rd_ptr==0);
endproperty

assert property (read_ptr_wrap_around_p);
cover property (read_ptr_wrap_around_p);

property count_wrap_around_p;
    @(posedge v_if.clk) ($past(count)==v_if.FIFO_DEPTH && !v_if.rst_n) |-> (count==0);
endproperty

assert property (count_wrap_around_p);
cover property (count_wrap_around_p);

property write_ptr_wrap_around_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
        (wr_ptr==v_if.FIFO_DEPTH-1) && !v_if.full && v_if.wr_en | => (wr_ptr==0);
endproperty

assert property (write_ptr_wrap_around_p);
cover property (write_ptr_wrap_around_p);

```

```

property wr_ptr_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
    (wr_ptr<v_if.FIFO_DEPTH);
endproperty

assert property (wr_ptr_p);
cover property (wr_ptr_p);

property rd_ptr_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
    (rd_ptr<v_if.FIFO_DEPTH);
endproperty

assert property (rd_ptr_p);
cover property (rd_ptr_p);

property count_p;
    @(posedge v_if.clk) disable iff(!v_if.rst_n)
    (count<=v_if.FIFO_DEPTH);
endproperty

assert property (count_p);
cover property (count_p);
endmodule

```

Verification Plan:

Feature	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
1. Initialization & Reset	Reset Behavior for internal signals and some outputs	Assert rst_n low (active low asynchronous reset); verify all internal counters and pointers reset to 0, and empty is asserted.	-	Assertion
2. Write Operation	Write Operation in memory	Assert wr_en when the FIFO is not full; verify data is written and internal count increments.	Cross(wr_en,rd_en,full)	Reference Model, Assertion
3. Read Operation	Assert rd_en when the FIFO is not empty; verify data is retrieved and internal count decrements.	Assert rd_en when the FIFO is not empty; verify data is retrieved and internal count decrements	By reduction XNOR inside the writing loop	Reference Model, Assertion
4. Data Integrity	Data Integrity (Implied)	Write a sequence of data to the FIFO, followed by a full read sequence; verify data_out matches the input sequence in FIFO order.	-	Reference Model (Data_out)
5. Full Flag	Full Flag Assertion	Write exactly FIFO_DEPTH entries; verify full asserts when count=FIFO_DEPTH	Cross(wr_en,rd_en,full)	Assertion
6. Almost Full (almostfull)	Almost Full Condition	Write exactly FIFO_DEPTH-1 entries; verify almostfull asserts.	Cross(wr_en,rd_en,almostfull)	Assertion
7. Empty Flag (empty)	Empty Flag Assertion	Read until count=0; verify empty asserts	Cross(wr_en,rd_en,empty)	Assertion
8. Almost Empty (almostempty)	Almost Empty Condition	Read until count=1; verify almostempty asserts.	Cross(wr_en,rd_en,almostempty)	Assertion
9. Overflow Condition	Overflow Detection	Assert wr_en when full is already asserted; verify overflow asserts and the write is non-destructive.	Cross(wr_en,rd_en,overflow)	Assertions
10. Underflow Condition	Underflow Detection	Assert rd_en when empty is already asserted; verify underflow asserts and the read is non-destructive.	Cross(wr_en,rd_en,underflow)	Assertions
11. Write Acknowledge	Write Acknowledg	Assert wr_en when not full; verify wr_ack asserts for one cycle confirming success.	Cross(wr_en,rd_en,wr_ack)	Assertion
12. Simultaneous R/W (Empty Priority)	Simultaneous R/W Priority (Empty)	Assert wr_en=1,rd_en=1 when empty=1; verify only the write succeeds (count increments by 1)	Cross(wr_en,rd_en,wr_ack), Cross(wr_en,rd_en,underflow)	Reference Model, Assertion
13. Simultaneous R/W (Full Priority)	Simultaneous R/W Priority (Full)	Assert wr_en=1,rd_en=1 when full=1; verify only the read succeeds (count decrements by 1).	Cross(wr_en,rd_en,full), Cross(wr_en,rd_en,overflow)	Reference Model, Assertion
14. Pointer Management	Pointer Wraparound , Pointer Threshold	Drive continuous writes/reads for more than FIFO_DEPTH cycles; check that pointers and counters correctly wrap around from Depth-1 to 0 and never exceed FIFO_DEPTH.	-	Assertion

Sync_FIFO_interface:

```
interface Sync_FIFO_interface(input bit clk);
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;

    bit [FIFO_WIDTH-1:0] data_in, data_out;
    bit rst_n, wr_en, rd_en, wr_ack, overflow, underflow,
        full, empty, almostfull, almostempty;
    event etrigger,btrigger;
    modport DUT (
        input data_in,clk, rst_n, wr_en, rd_en,
        output data_out, wr_ack, overflow, underflow, full, empty,
            almostfull, almostempty
    );

    modport TB (
        input clk, etrigger,btrigger, data_out, wr_ack, overflow,
            underflow, full, empty, almostfull, almostempty,
        output data_in, rst_n, wr_en, rd_en
    );

    modport MON (
        input clk, data_out, wr_ack, overflow, underflow, full,
            empty, almostfull, almostempty, data_in, rst_n,
            wr_en, rd_en, etrigger,btrigger
    );
endinterface
```

shared_pkg.:

```
package shared_pkg;
    bit test_finished=0;
    int correct_count=0,errors_count=0;
endpackage
```

FIFO_transaction:

```
package FIFO_transaction;
    class FIFO_transaction #(parameter FIFO_WIDTH=16, parameter FIFO_DEPTH=8);

        rand bit [FIFO_WIDTH-1:0] data_in;
        rand bit rst_n, wr_en, rd_en;
        int RD_EN_ON_DIST, WR_EN_ON_DIST;
        logic [FIFO_WIDTH-1:0] data_out;
        bit wr_ack, overflow, underflow, full, empty, almostfull, almostempty;

        function new (input int RD_EN_ON_DIST=30,input int WR_EN_ON_DIST=70);
            this.RD_EN_ON_DIST=RD_EN_ON_DIST;
            this.WR_EN_ON_DIST=WR_EN_ON_DIST;
        endfunction
    endclass
endpackage
```



```

endfunction

constraint randoms{
    rst_n dist{ 1 :/98, 0 :/2 };
    wr_en dist{ 1:/WR_EN_ON_DIST, 0:/(100-WR_EN_ON_DIST) };
    rd_en dist{ 1:/RD_EN_ON_DIST, 0:/(100-RD_EN_ON_DIST) };
}
endclass
endpackage

```

FIFO_scoreboard:

```

package FIFO_scoreboard;
import FIFO_transaction::*;
import shared_pkg::*;

class FIFO_scoreboard #(parameter FIFO_WIDTH=16, parameter FIFO_DEPTH=8);
    FIFO_transaction trans_inputs;
    logic [FIFO_WIDTH-1:0] data_out_ref;
    bit wr_ack_ref, overflow_ref, underflow_ref, full_ref, empty_ref, almostfull_ref,
almostempty_ref;
    logic [FIFO_WIDTH-1:0] mem_q[$];
    function void ref_model(input FIFO_transaction trans_inputs);
        if (!trans_inputs.rst_n) begin
            mem_q.delete();
        end
        else begin
            if ({trans_inputs.wr_en,trans_inputs.rd_en}==2'b11) begin
                if (mem_q.size()==0)
                    mem_q.push_back(trans_inputs.data_in);
                else if(mem_q.size()==8)
                    data_out_ref = mem_q.pop_front();
                else begin
                    mem_q.push_back(trans_inputs.data_in);
                    data_out_ref = mem_q.pop_front();
                end
            end
            else begin
                if (trans_inputs.wr_en && mem_q.size() != 8)
                    mem_q.push_back(trans_inputs.data_in);
                else if (trans_inputs.rd_en && (mem_q.size() != 0))
                    data_out_ref = mem_q.pop_front();
            end
        end
    endfunction

    function void check_data(input FIFO_transaction trans_inputs);
        ref_model(trans_inputs);
        if (trans_inputs.data_out!=data_out_ref) begin
            $display("Test failed, error here Data_out = %h, Expected = %h, rd_en =
%b, wr_en = %b",trans_inputs.data_out,data_out_ref,
trans_inputs.rd_en,trans_inputs.wr_en);

```

```

        errors_count++;
    end else correct_count++;
endfunction
endclass
endpackage

```

FIFO_coverage:

```

package FIFO_coverage;
import FIFO_transaction::*;
import shared_pkg::*;
class FIFO_coverage;
    FIFO_transaction F_cvg_txn;
    covergroup FIFO_cg;
        cp_write_en    : coverpoint F_cvg_txn.wr_en          {option.weight=0;bins
wr_en_zero = { 1'b0 };bins wr_active  = { 1'b1 }; }
        cp_read_en     : coverpoint F_cvg_txn.rd_en          {option.weight=0; bins
rd_en_zero = { 1'b0 }; bins rd_active = { 1'b1 };}
        cp_full        : coverpoint F_cvg_txn.full           {option.weight=0; bins
full_asserted  = { 1'b1 }; bins full_deasserted = { 1'b0 }; }
        cp_empty       : coverpoint F_cvg_txn.empty          {option.weight=0;}
        cp_almost_full : coverpoint F_cvg_txn.almostfull     {option.weight=0;}
        cp_almost_empty : coverpoint F_cvg_txn.almostempty   {option.weight=0;}
        cp_overflow    : coverpoint F_cvg_txn.overflow       {option.weight=0; bins
overflow_asserted  = { 1'b1 }; bins overflow_deasserted = { 1'b0 };}
        cp_underflow   : coverpoint F_cvg_txn.underflow      {option.weight=0; bins
underflow_asserted  = { 1'b1 }; bins underflow_deasserted = { 1'b0 };}
        cp_wr_ack      : coverpoint F_cvg_txn.wr_ack         {option.weight=0; bins
ack_asserted   = { 1'b1 }; bins ack_deasserted  = { 1'b0 };}
        /////////////// Cross
        full_cv        : cross cp_write_en, cp_read_en, cp_full {
            ignore_bins redundant_full =
                binsof(cp_read_en.rd_active) &&
                binsof(cp_full.full_asserted); }
        empty_cv       : cross cp_write_en, cp_read_en, cp_empty;
        almostempty_cv : cross cp_write_en, cp_read_en, cp_almost_empty;
        almostfull_cv  : cross cp_write_en, cp_read_en, cp_almost_full;
        overflow_cv     : cross cp_write_en, cp_read_en, cp_overflow{
            ignore_bins impossible_overflow =
                binsof(cp_write_en.wr_en_zero) &&
                binsof(cp_overflow.overflow_asserted); }
        underflow_cv   : cross cp_write_en, cp_read_en, cp_underflow{
            ignore_bins impossible_underflow =
                binsof(cp_read_en.rd_en_zero) &&
                binsof(cp_underflow.underflow_asserted);}
        wr_ack_cv      : cross cp_write_en, cp_read_en, cp_wr_ack{
            ignore_bins impossible_wr_ack =
                binsof(cp_write_en.wr_en_zero) &&
                binsof(cp_wr_ack.ack_asserted);}

    endgroup
    function new();
        FIFO_cg=new;

```

```

        endfunction
        function void sample_data(input FIFO_transaction F_txn);
            F_cvg_txn=F_txn;
            FIFO_cg.sample();
        endfunction
    endclass
endpackage

```

FIFO_monitor:

```

import FIFO_coverage::*;
import FIFO_transaction::*;
import FIFO_scoreboard::*;
import shared_pkg::*;
module FIFO_monitor(Sync_FIFO_interface.MON v_if);
    FIFO_coverage cov;
    FIFO_scoreboard sb;
    FIFO_transaction trans;
    initial begin
        trans=new();
        cov=new();
        sb=new();
        forever begin
            @(negedge v_if.clk);
            wait(v_if.etrigger.triggered);
            trans.data_in=v_if.data_in;
            trans.wr_ack=v_if.wr_ack;
            trans.rd_en=v_if.rd_en;
            trans.wr_en=v_if.wr_en;
            trans.overflow=v_if.overflow;
            trans.underflow=v_if.underflow;
            trans.full=v_if.full;
            trans.empty=v_if.empty;
            trans.almostempty=v_if.almostempty;
            trans.almostfull=v_if.almostfull;
            trans.data_out=v_if.data_out;
            trans.rst_n=v_if.rst_n;
            -> v_if.btrigger;
            fork
                begin
                    cov.sample_data(trans);
                end
                begin
                    sb.check_data(trans);
                end
            join
            if (test_finished) begin
                $display("correct_count = %d, errors_count = %d",
                    correct_count,errors_count);
                $stop;
            end
        end
    end
endmodule

```

FIFO_TB:

```
import FIFO_coverage::*;
import FIFO_transaction::*;
import FIFO_scoreboard::*;
import shared_pkg::*;
module FIFO_TB(Sync_FIFO_interface.TB v_if);
FIFO_transaction trans;
    int stopped=79999;
    initial begin
        trans =new();
        v_if.rst_n=0;
        @(negedge v_if.clk);
        v_if.rst_n=1;
        @(negedge v_if.clk);
        repeat (10000) begin
            v_if.rd_en = 0; v_if.wr_en = 1; v_if.rst_n = 1;
            assert(trans.randomize());
            v_if.data_in = trans.data_in;
            @(negedge v_if.clk);
            ->v_if.etrigger;
            wait ( v_if.btrigger.triggered);
        end
        repeat (10000) begin
            v_if.rd_en = 1; v_if.wr_en = 1; v_if.rst_n = 1;
            assert(trans.randomize());
            v_if.data_in = trans.data_in;
            @(negedge v_if.clk);
            ->v_if.etrigger;
            wait ( v_if.btrigger.triggered);
        end
        repeat (10000) begin
            v_if.rd_en = 1; v_if.wr_en = 0; v_if.rst_n = 1;
            assert(trans.randomize());
            v_if.data_in = trans.data_in;
            @(negedge v_if.clk);
            ->v_if.etrigger;
            wait ( v_if.btrigger.triggered);
        end
        for (int i=0; i<=stopped; ++i) begin
            assert(trans.randomize());
            v_if.data_in = trans.data_in;
            v_if.rd_en = trans.rd_en;
            v_if.wr_en = trans.wr_en;
            v_if.rst_n = trans.rst_n;
            @(negedge v_if.clk);
            ->v_if.etrigger;
            if (i != stopped) wait ( v_if.btrigger.triggered);
        end
        test_finished = 1;
    end
endmodule
```

FIFO_top:

```
module top();
    bit clk;
initial begin
    clk=0;
    forever #4 clk=~clk;
end
    Sync_FIFO_interface v_if (clk);
    FIFO DUT(v_if);
    FIFO_TB TB(v_if);
    FIFO_monitor MON(v_if);
property reset_p;
    @(posedge v_if.clk) !v_if.rst_n
    |-> ({v_if.overflow, v_if.underflow} == 0 )
endproperty
assert property(reset_p);
cover property(reset_p);

endmodule
```

Do file:

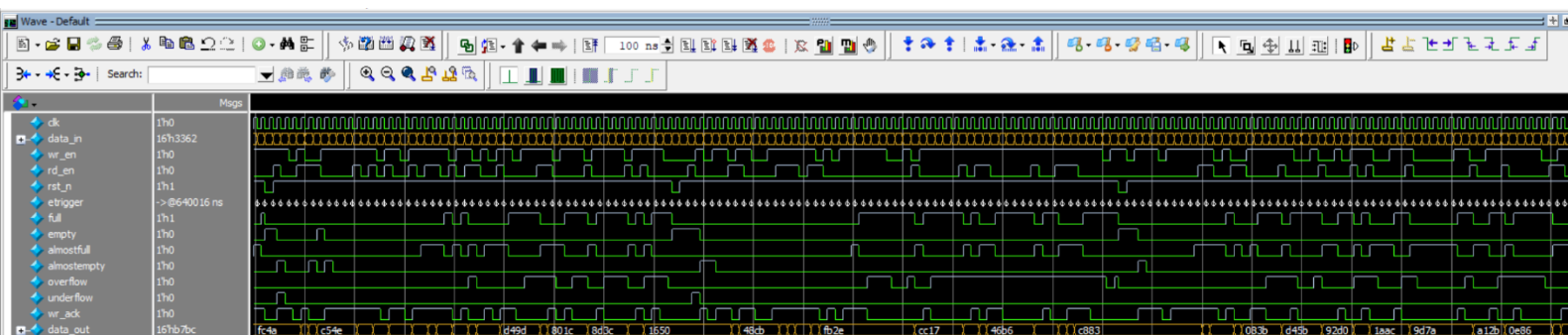
F: > KareemWaseem > Verification_UVM > project_1 > run_FIFO.do

```
1 vlib work
2 vlog -f src_files.list +cover -covercells
3 vsim -voptargs=+acc work.top -cover
4 add wave *
5 add wave -position insertpoint \
6 sim:/top/v_if/data_in \
7 sim:/top/v_if/data_out \
8 sim:/top/v_if/wr_en \
9 sim:/top/v_if/rd_en \
10 sim:/top/v_if/rst_n \
11 sim:/top/v_if/full \
12 sim:/top/v_if/almostfull \
13 sim:/top/v_if/empty \
14 sim:/top/v_if/almostempty \
15 sim:/top/v_if/overflow \
16 sim:/top/v_if/underflow \
17 sim:/top/v_if/wr_ack \
18 sim:/top/v_if/etrigger
19 coverage save top.ucdb -onexit
20 run 0
21 add wave -position insertpoint sim:/top/v_if/*
22 run -all
```

F: > KareemWaseem > Verification_UVM > project_1 > src_files.list

```
1 FIFO.sv
2 Sync_FIFO_interface.sv
3 shared_pkg.sv
4 FIFO_transaction.sv
5 FIFO_scoreboard.sv
6 FIFO_coverage.sv
7 FIFO_monitor.sv
8 FIFO_TB.sv
9 FIFO_top.sv
```

Wave form:



Assertions:

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active C	Memory	Peak Mem	Peak Mem	Cumula	ATV	Assertion Expression	Included
/top/assert_reset_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) (~v_if...	✓
/top/DUT/assert_reset_internal	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(negedge v_if.clk) (~v_if...	✓
/top/DUT/assert_write_acknowledge_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_overflow_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_underflow_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_empty_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_full_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_almostfull_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_almostempty_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_read_ptr_wrap_around_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_count_wrap_around_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) ((\$past...	✓
/top/DUT/assert_write_ptr_wrap_around_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_wr_ptr_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_rd_ptr_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/DUT/assert_count_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if.clk) disable...	✓
/top/TB/#ublk#182146242#14/immed__16	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert(trans.randomize())	✓
/top/TB/#ublk#182146242#22/immed__24	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert(trans.randomize())	✓
/top/TB/#ublk#182146242#30/immed__32	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert(trans.randomize())	✓
/top/TB/#anonblk#182146242#38#4#/#ublk#182146242#38/...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert(trans.randomize())	✓

Functional Coverage:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/FIFO_coverage/FIFO_coverage		100.00%							
TYPE FIFO_cg		100.00%	100	100.00...		✓	auto(1)		
+ CVP FIFO_cg::cp_write_en		100.00%	100	100.00...		✓			
+ CVP FIFO_cg::cp_read_en		100.00%	100	100.00...		✓			
+ CVP FIFO_cg::cp_full		100.00%	100	100.00...		✓			
+ CVP FIFO_cg::cp_empty		100.00%	100	100.00...		✓			
+ CVP FIFO_cg::cp_almost_full		100.00%	100	100.00...		✓			
+ CVP FIFO_cg::cp_almost_empty		100.00%	100	100.00...		✓			
+ CVP FIFO_cg::cp_overflow		100.00%	100	100.00...		✓			
+ CVP FIFO_cg::cp_underflow		100.00%	100	100.00...		✓			
+ CVP FIFO_cg::cp_wr_ack		100.00%	100	100.00...		✓			
+ CROSS FIFO_cg::full_cv		100.00%	100	100.00...		✓			
+ CROSS FIFO_cg::empty_cv		100.00%	100	100.00...		✓			
+ CROSS FIFO_cg::almostempty_cv		100.00%	100	100.00...		✓			
+ CROSS FIFO_cg::almostfull_cv		100.00%	100	100.00...		✓			
+ CROSS FIFO_cg::overflow_cv		100.00%	100	100.00...		✓			
+ CROSS FIFO_cg::underflow_cv		100.00%	100	100.00...		✓			
+ CROSS FIFO_cg::wr_ack_cv		100.00%	100	100.00...		✓			

Code coverage:

Code Coverage Analysis

Statements - by instance (/top/DUT)

FIFO.sv

✓

9

always @(posedge v_if.clk or negedge v_if.rst_n) begin

✓

11

wr_ptr <= 0;

✓

12

v_if.wr_ack <= 0; //was bug

✓

13

v_if.overflow <= 0; //was bug

✓

16

mem[wr_ptr] <= v_if.data_in;

✓

17

v_if.wr_ack <= 1;

✓

18

wr_ptr <= wr_ptr + 1;

✓

21

v_if.wr_ack <= 0;

✓

23

v_if.overflow <= 1;

✓

25

v_if.overflow <= 0;

✓

28

always @(posedge v_if.clk or negedge v_if.rst_n) begin

✓

30

rd_ptr <= 0;

✓

31

v_if.underflow <=0; //was bug

✓

34

v_if.data_out <= mem[rd_ptr];

✓

35

rd_ptr <= rd_ptr + 1;

✓

39

v_if.underflow <= 1;

✓

41

v_if.underflow <=0;

✓

44

always @(posedge v_if.clk or negedge v_if.rst_n) begin

✓

46

count <= 0;

✓

50

count <= count + 1;

✓

52

count <= count - 1;

✓

54

count <= count-1;

✓

56

count <= count+1;

✓

60

assign v_if.full = (count == v_if.FIFO_DEPTH)? 1 : 0;

✓

61

assign v_if.empty = (count == 0)? 1 : 0;

✓

62

assign v_if.almostfull = (count == v_if.FIFO_DEPTH-1)? 1 : 0; //was bug

✓

63

assign v if.almostempty = (count == 1)? 1 : 0;

B Code Coverage Analysis

Branches - by instance (/top/DUT)

FIFO.sv

- 10 if (!v_if.rst_n) begin
- 15 else if (v_if.wr_en && !v_if.full) begin
- 20 else begin
- 22 if (v_if.full && v_if.wr_en)
- 24 else
- 29 if (!v_if.rst_n) begin
- 33 else if (v_if.rd_en && !v_if.empty) begin
- 37 else begin //was bug
- 38 if (v_if.empty && v_if.rd_en)
- 40 else
- 45 if (!v_if.rst_n) begin
- 48 else begin
- 49 if (({v_if.wr_en, v_if.rd_en} == 2'b10) && !v_if.full)
- 51 else if (({v_if.wr_en, v_if.rd_en} == 2'b01) && !v_if.empty)
- 53 else if(({v_if.wr_en, v_if.rd_en} == 2'b11) && v_if.full) //was bug
- 55 else if(({v_if.wr_en, v_if.rd_en} == 2'b11) && v_if.empty) //bug continue
- 60 assign v_if.full = (count == v_if.FIFO_DEPTH)? 1 : 0;
- 61 assign v_if.empty = (count == 0)? 1 : 0;
- 62 assign v_if.almostfull = (count == v_if.FIFO_DEPTH-1)? 1 : 0; //was bug
- 63 assign v_if.almostempty = (count == 1)? 1 : 0;

C Code Coverage Analysis

Conditions - by instance (/top/DUT)

FIFO.sv

- 15 else if (v_if.wr_en && !v_if.full) begin
- 22 if (v_if.full && v_if.wr_en)
- 33 else if (v_if.rd_en && !v_if.empty) begin
- 38 if (v_if.empty && v_if.rd_en)
- 49 if (({v_if.wr_en, v_if.rd_en} == 2'b10) && !v_if.full)
- 51 else if (({v_if.wr_en, v_if.rd_en} == 2'b01) && !v_if.empty)
- 53 else if(({v_if.wr_en, v_if.rd_en} == 2'b11) && v_if.full) //was bug
- 55 else if(({v_if.wr_en, v_if.rd_en} == 2'b11) && v_if.empty) //bug continue
- 60 assign v_if.full = (count == v_if.FIFO_DEPTH)? 1 : 0;
- 61 assign v_if.empty = (count == 0)? 1 : 0;
- 62 assign v_if.almostfull = (count == v_if.FIFO_DEPTH-1)? 1 : 0; //was bug
- 63 assign v_if.almostempty = (count == 1)? 1 : 0;

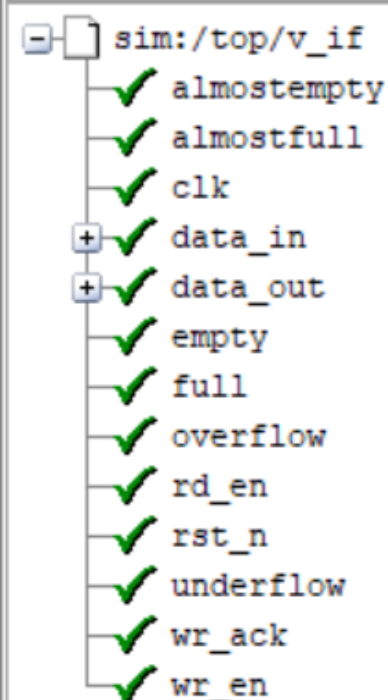
T Code Coverage Analysis

Toggles - by instance (/top/DUT)

sim:/top/DUT

- count
- rd_ptr
- wr_ptr

Toggles - by instance (/top/v_if)



Cover Directives

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included
🔗 /top/cover__reset_p	SVA	✓	Off	1565	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__count_p	SVA	✓	Off	08437	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__rd_ptr_p	SVA	✓	Off	08437	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__wr_ptr_p	SVA	✓	Off	08437	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__write_ptr_wrap_around_p	SVA	✓	Off	4643	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__count_wrap_around_p	SVA	✓	Off	628	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__read_ptr_wrap_around_p	SVA	✓	Off	3483	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__almostempty_p	SVA	✓	Off	3486	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__almostfull_p	SVA	✓	Off	31302	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__full_p	SVA	✓	Off	41809	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__empty_p	SVA	✓	Off	12601	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__underflow_p	SVA	✓	Off	10742	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__overflow_p	SVA	✓	Off	31709	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__write_acknowledge_p	SVA	✓	Off	41965	1	Unli...	1	100%		✓
🔗 /top/DUT/cover__reset_internal	SVA	✓	Off	1565	1	Unli...	1	100%		✓

ALL FILES IN THE ZIP. FILE.